

SFT-GRPO Data Overlap as a Post-Training Hyperparameter for Autoformalization

XIAOLE SU KASEY ZHANG ANDY LYU

Osmosis AI

xiaole@osmosis.ai kasey@osmosis.ai andy@osmosis.ai

Abstract

Supervised fine-tuning (SFT) followed by Group Relative Policy Optimization (GRPO) is a common post-training recipe. We conduct a controlled ablation over SFT-GRPO data overlap, evaluating Qwen3-8B (thinking disabled) post-trained for Lean 4 autoformalization under six conditions that differ solely in training recipe: a base model, SFT-only, GRPO-only, and three SFT+GRPO configurations where 0%, 30%, or 100% of the GRPO prompts coincide with the SFT corpus. Keeping SFT and GRPO data disjoint consistently outperforms full overlap at zero additional compute cost. Evaluating on Gaokao-Formal and PutnamBench under both compile pass@ k and semantic pass@ k (assessed by an LLM judge), we find that: (1) lower overlap is monotonically associated with higher compilation and semantic accuracy; (2) at 0% overlap, GRPO yields a 10.4 percentage-point (pp) semantic gain over SFT alone on Gaokao; (3) at 100% overlap, both compilation and semantic accuracy remain flat, rendering the GRPO stage effectively redundant; (4) dual-metric evaluation reveals compile-semantic gaps exceeding 30 pp for the highest-compiling models, a disparity invisible under compile-only benchmarking. To our knowledge, this is the first controlled investigation of SFT-GRPO data overlap as a post-training hyperparameter, showing how model behavior varies based on the degree of data sharing between stages.

1 Introduction

Autoformalization, the translation of informal mathematics into machine-verifiable formal languages, is a prerequisite for scaling formal verification to the breadth of human mathematical knowledge [Wu et al., 2022]. Lean 4 [de Moura et al., 2021] and its mathematical library Mathlib have emerged as the dominant target, with recent systems achieving high compilation rates on standard benchmarks [Wang et al., 2025, Zhang et al., 2025, Wu et al., 2025].

Prevailing approaches combine supervised fine-tuning (SFT) on labeled (natural-language, Lean 4) pairs with reinforcement learning, typically Group Relative Policy Optimization (GRPO) [Shao et al., 2024], using compiler feedback as part of the reward signal. While the SFT-GRPO pipeline has become standard practice, a fundamental design decision remains underexplored: most published pipelines draw their GRPO corpus from the same pool as the SFT data (or use identical data) with no explicit consideration of overlap. We investigate how the degree of training data overlap between the SFT and GRPO stages affects downstream formalization quality. Concurrently, compilation pass rate remains the predominant evaluation metric in autoformalization studies [Wang et al., 2025, Huang et al., 2025], often leaving open whether compiling outputs faithfully capture the semantics of the original mathematical statement.

This paper reports a controlled study of SFT-GRPO data overlap as a post-training hyperparameter for Lean 4 autoformalization. Holding the base model (Qwen3-8B, thinking disabled), data sources, and all other hyperparameters fixed, we evaluate six conditions differing solely in training recipe and in the fraction of GRPO prompts that overlap with the SFT corpus (0%, 30%, 100%). We evaluate on Gaokao-Formal [Zhang et al., 2024] (495 problems, moderate

Natural-language problem (Putnam 1962 A6):

Let S be a set of rational numbers closed under addition and multiplication, with the property that for every rational r exactly one of $r \in S$, $-r \in S$, $r = 0$ holds. Prove that S is the set of all positive rationals.

Lean 4 formalization:

```

theorem putnam_1962_a6 (S : Set ℚ)
  (hSadd : ∀ a ∈ S, ∀ b ∈ S, a + b ∈ S)
  (hSprod : ∀ a ∈ S, ∀ b ∈ S, a * b ∈ S)
  (hScond : ∀ r : ℚ, (r ∈ S ∨ -r ∈ S ∨ r = 0)
    ∧ ¬(r ∈ S ∧ -r ∈ S)
    ∧ ¬(r ∈ S ∧ r = 0)
    ∧ ¬(-r ∈ S ∧ r = 0))
  : S = { r : ℚ | r > 0 } := sorry

```

Figure 1: The autoformalization task: translate a natural-language mathematical statement into a Lean 4 theorem declaration that typechecks against Mathlib. The model must encode the problem’s hypotheses and conclusion as typed Lean 4 expressions; `sorry` stands in for the proof (§3.1).

difficulty) and PutnamBench [Tsoukalas et al., 2024] (672 problems, high difficulty) using both compile pass@ k and semantic pass@ k , where the latter quantifies semantic faithfulness using an LLM judge (threshold ≥ 0.7).

Our principal contributions are:

1. **Controlled ablation of SFT-GRPO data overlap.** Lower overlap is monotonically associated with higher compilation and semantic accuracy. At 0% overlap, GRPO yields a 10.4 pp semantic gain over SFT alone on Gaokao. At 100% overlap, both metrics remain flat, indicating that non-overlapping data is a necessary condition for GRPO to confer meaningful benefit.
2. **Dual-metric evaluation.** We evaluate using both compile pass@ k and semantic pass@ k , revealing compile-semantic gaps exceeding 30 pp for the highest-compiling models. These gaps are invisible under compile-only evaluation.

2 Background and Related Work

Autoformalization. Autoformalization encompasses the full translation of informal mathematics into formal, machine-verifiable languages, covering both problem statements and proofs [Wu et al., 2022, Weng et al., 2025]. Within this broad endeavor, *statement formalization* is a distinct subtask: translating a natural-language theorem statement into a formal declaration without producing a proof. Accurate statement formalization serves as a critical starting point for downstream *prover models* such as Kimina-Prover [Wang et al., 2025], DeepSeek-Prover-V2 [Xin et al., 2025], and Goedel-Prover [Lin et al., 2025], which handle tactic generation and proof search.

Recent training approaches for statement formalization span a broad spectrum: SFT on distilled data [Wang et al., 2025], SFT combined with RL under various reward designs [Zhang et al., 2025, Wu et al., 2025], pure RL without labeled data [Huang et al., 2025], and retrieval-augmented or chain-of-thought prompting strategies [Weng et al., 2025]. A key design axis across these systems is the reward signal for RL. FormaRL [Huang et al., 2025] uses a binary reward combining compilation success and LLM consistency checking. StepFun [Wu et al., 2025] employs Bidirectional Extended Definitional Equivalence (BEq) as a formally grounded reward.

Table 1: Source quality: 500-sample compilation test against Mathlib v4.27.0.

Dataset	Pool Size	Pass Rate
Leanabell-Prover [stoney0062, 2025]	1.13M	93.0%
Lean Workbook [He et al., 2024]	13.5K	92.0%
NuminaMath [Li et al., 2024]	104K	90.6%
HERALD [Gao et al., 2025]	580K	86.6%

Mathesis [Zhang et al., 2025] introduces LeanScorer for partial-credit feedback. Our GRPO configuration is closest to FormaRL’s but augments the reward with a continuous semantic judge score rather than relying solely on binary feedback.

SFT-RL interaction. DeepSeek-R1 [DeepSeek-AI, 2025] demonstrates that RL can generalize beyond the SFT training distribution, and Chu et al. [2025] provide complementary evidence, showing that SFT tends to memorize training distributions while RL generalizes beyond them. Lu et al. [2026] advance a data-centric perspective on RLHF, arguing that RL’s advantage derives partly from implicit data curation through the reward signal. Our work extends this line of inquiry by isolating a specific data-management variable, the overlap between SFT and GRPO pools, and quantifying its effect on both compilation and semantic accuracy.

Evaluation beyond compilation. Most autoformalization work evaluates only whether generated statements typecheck [Wang et al., 2025, Huang et al., 2025]. BEq provides the strongest semantic guarantee available, formally verifying logical equivalence through bidirectional proof construction, though it suffers from high false-negative rates when the prover cannot close equivalent but syntactically dissimilar goals [Wu et al., 2025, Weng et al., 2025]. LeanScorer offers a more granular partial-credit assessment [Zhang et al., 2025]. LLM-judge approaches trade formal guarantees for scalability and coverage; Weng et al. [2025] identify semantic evaluation as an open challenge for the field. We adopt an LLM judge as a scalable proxy (§5).

3 Data

3.1 Task Definition

We study *autoformalization*: translating natural-language mathematical statements into Lean 4 theorem declarations that typecheck against Mathlib v4.27.0. Each output is a standalone theorem terminated by `:= by sorry`, where `sorry` is a Lean 4 keyword that accepts any goal without a proof, allowing the statement to typecheck independently of proof search. No proof body is required; this formulation isolates statement formalization from proving.

3.2 Sources and Processing

We draw from four publicly available corpora: **NuminaMath** [Li et al., 2024] (104K competition math problems), **Leanabell-Prover** [stoney0062, 2025] (1.13M aggregated entries), **HERALD** [Gao et al., 2025] (580K NL–FL pairs from Mathlib4), and **Lean Workbook** [He et al., 2024] (13.5K contest-style pairs). Each source was vetted by compiling a 500-sample slice against Mathlib v4.27.0 (Table 1).

The curation pipeline comprises seven stages: ingestion, compilation (against Mathlib v4.27.0), deduplication (SHA-256, essential because Leanabell aggregates 15–20% duplicate content from NuminaMath and Lean Workbook), quality filtering, topic stratification, answer injection (§3.5), and formatting to Alpaca JSONL.

Table 2: SFT dataset (20K) composition.

Source	Count	%	Role
NuminaMath	8,000	40%	Topic-balanced competition math
Leanabell	7,000	35%	Olympiad + AoPS diversity
HERALD	3,000	15%	Broad Mathlib coverage
Lean Workbook	2,000	10%	Proved + disproved statements
Total	20,000	100%	

3.3 SFT Dataset

The SFT dataset consists of 20,000 (natural-language, Lean 4) pairs, composed as shown in Table 2.

3.4 GRPO Dataset and Overlap Definition

The GRPO corpus consists of $\sim 16,000$ natural-language prompts sampled from the same four sources. Each prompt is paired with a ground-truth Lean 4 statement consumed exclusively by the reward function; the model never observes it during generation. We define *overlap* as the fraction of GRPO prompts that also appeared in the SFT training set:

- **0% overlap:** all 16K prompts are fresh samples, cross-deduplicated against the SFT pool.
- **30% overlap:** $\sim 4.8K$ prompts are resampled from the SFT pool; the remaining $\sim 11.2K$ are fresh.
- **100% overlap:** the entire GRPO corpus is drawn from the SFT pool.

All three pools share the same source distribution and differ only in their intersection with the SFT set, isolating overlap as the sole experimental variable. Because the pools are drawn from the same four sources with comparable quality profiles (Table 1), we expect difficulty distributions to be broadly similar across overlap conditions, though we do not formally verify this.

3.5 Answer Injection

Many competition problems require the model to “find” or “determine” a value that appears only in the ground-truth Lean 4 statement. A formalization model is not expected to solve the underlying math problem; its task is translation, not computation. However, during GRPO the model must produce a complete formalization from the prompt alone. Without the target value, the model cannot formalize a correct statement because it simply does not know the correct answer. We observe that GRPO-only training without answer injection produces consistently poor formalizations for such problems.

Figure 2 illustrates this with a concrete example from LeanWorkbook. Without injection, the model must independently compute that the series $\sum_{k=1}^{\infty} k^2/2^k$ converges to 6 before it can even begin formalizing. Our answer-injection pipeline extracts concrete answers from the Lean 4 ground truth using 12 pattern types and appends them to “find”-type prompts (e.g., “Show that the answer is 6”), so the model can focus entirely on formalization rather than re-deriving the answer. This pipeline is particularly relevant for “find the value” and “determine the answer” problems, which form a substantial fraction of competition mathematics datasets. Most existing autoformalization datasets are designed for SFT with ground-truth outputs, not for RL where the model generates from prompts alone. For problems of this type, answer injection bridges the

Without answer injection:

Prompt: “Determine the value of the infinite series $\sum_{k=1}^{\infty} \frac{k^2}{2^k}$.”

Model must: (1) compute $\sum k^2/2^k = 6$, then (2) formalize as Lean 4.

With answer injection:

Prompt: “Determine the value of the infinite series $\sum_{k=1}^{\infty} \frac{k^2}{2^k}$. Show that the answer is 6.”

Model must: (1) formalize the statement directly.

Lean 4 output:

```
theorem lean_workbook :
   $\sum' k : \mathbb{N}, (k \wedge 2 : \mathbb{R}) / 2 \wedge k = 6 :=$  by sorry
```

Figure 2: Answer injection eliminates the need for the model to solve the underlying math problem during GRPO. Without the injected answer, the model does not know the correct value and cannot formalize a correct statement.

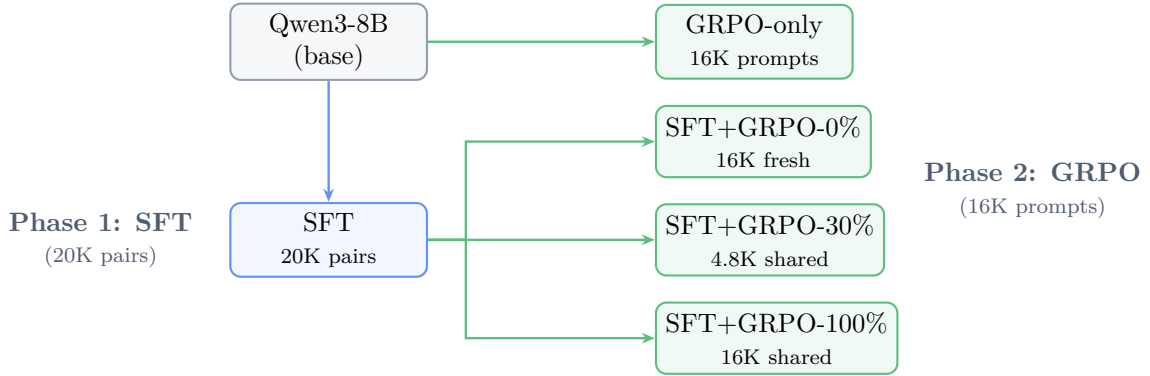


Figure 3: Training overview. Phase 1 produces one SFT checkpoint. Phase 2 applies GRPO to both the base model (producing GRPO-only) and the SFT checkpoint (producing three overlap variants). All Phase 2 runs share identical hyperparameters; only the data pool and initialization differ.

gap, enabling GRPO training on standard SFT datasets without requiring the model to solve the underlying mathematics.

4 Training

We use Qwen3-8B [Qwen Team, 2025] as the base model with thinking mode **disabled** (i.e., `enable_thinking=False` in the Qwen3 chat template, suppressing internal chain-of-thought generation) for all conditions. No publicly available dataset provides ground-truth reasoning traces for Lean 4 autoformalization, and enabling chain-of-thought (whether natively or through distillation) would introduce an additional confounding variable, preventing attribution of observed gains to the training recipe alone. Training proceeds in two phases, yielding six conditions (Figure 3).

4.1 Phase 1: Supervised Fine-Tuning

Qwen3-8B is fine-tuned on the 20K SFT corpus using Axolotl with full-parameter updates, a cosine learning rate schedule (peak 2×10^{-5}), and 2 epochs. Full hyperparameters are provided in Table 8.

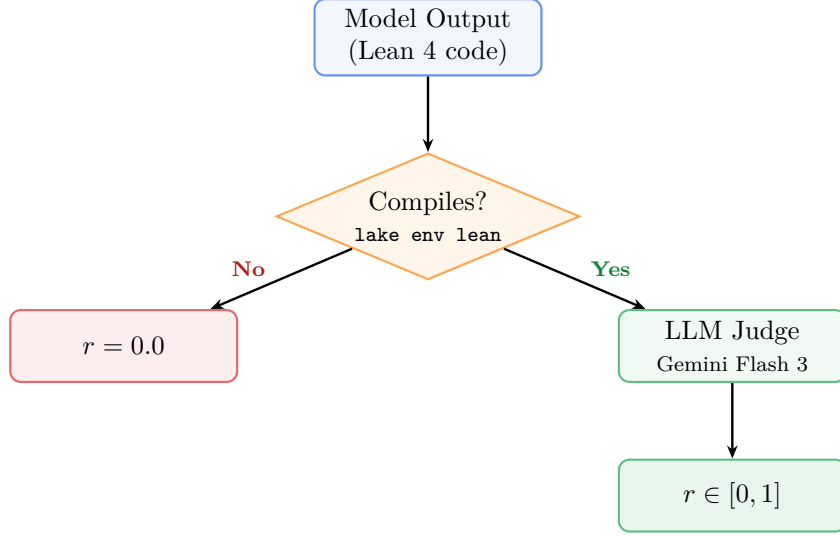


Figure 4: Dual-stage reward function. Compilable outputs are scored by Gemini Flash 3 for semantic faithfulness; non-compiling outputs receive zero reward.

Algorithm 1 Dual-Stage Reward

Require: NL input x , model output $\hat{y}$, ground truth y^*

Ensure: Reward $r \in [0, 1]$

- 1: $(ok, errors) \leftarrow \text{LEANTYPECHECK}(\hat{y}, \text{Mathlib v4.27.0})$
 - 2: **if** $\neg ok$ **then**
 - 3: **return** $r \leftarrow 0.0$
 - 4: **end if**
 - 5: $s_{\text{sem}} \leftarrow \text{GEMINIFLASH3JUDGE}(x, \hat{y}, y^*)$
 - 6: **return** $r \leftarrow s_{\text{sem}}$
-

4.2 Phase 2: Group Relative Policy Optimization

GRPO [Shao et al., 2024] is implemented using Slime/Megatron-LM with 8 rollouts per prompt and a constant learning rate of 1×10^{-6} . All four GRPO runs (three overlap variants and GRPO-only) share identical hyperparameters; only the initialization (base vs. SFT checkpoint) and data pool differ. Full hyperparameters are provided in Table 9.

4.3 Reward Function

The reward function couples a hard compiler gate with a continuous semantic judge (Figure 4, Algorithm 1). Outputs that fail to typecheck against Mathlib v4.27.0 receive $r=0$. Compiling outputs are scored by Gemini Flash 3, which assesses semantic faithfulness against the ground-truth Lean 4 statement on a continuous scale $r \in [0, 1]$, where 1.0 denotes a semantically equivalent formalization. This dual-stage design furnishes GRPO with indirect semantic supervision: the model never observes the ground-truth output, yet receives graded feedback on the quality of its own generations.

A single system prompt is shared across SFT training, GRPO training, and evaluation (Section D).

5 Evaluation

5.1 Benchmarks

We evaluate on two benchmarks spanning distinct difficulty regimes:

- **Gaokao-Formal** [Zhang et al., 2024]: 495 problems drawn from the Chinese national college entrance examination (Gaokao), spanning algebra, analysis, and number theory. Moderate difficulty.
- **PutnamBench** [Tsoukalas et al., 2024]: 672 problems from the William Lowell Putnam Mathematical Competition, covering a broad range of undergraduate and competition-level mathematics. High difficulty.

5.2 Metrics

We report two complementary $\text{pass}@k$ metrics with $n=8$ rollouts per problem:

- **Compile $\text{pass}@k$ ($C@k$)**: a rollout counts as a success if it typechecks against Mathlib v4.27.0.
- **Semantic $\text{pass}@k$ ($S@k$)**: a rollout counts as a success only if it compiles *and* receives a semantic faithfulness score ≥ 0.7 from the judge. This is the primary quality metric.

The **compile-semantic gap** ($C@k - S@k$) measures how often a model produces syntactically valid but semantically incorrect formalizations.

5.3 Semantic Judge

Each compiling output is scored by Gemini Flash 3 (temperature 0.0, max 1024 output tokens) on a continuous 0.0–1.0 scale for semantic faithfulness to the ground-truth formalization. Non-compiling outputs receive 0.0. The full judge prompt is provided in Section E.

5.4 External Baselines

We include two contemporary specialized autoformalizers evaluated under the same harness for contextual comparison: **Kimina-Autoformalizer-7B** [Wang et al., 2025] and **Mathesis-7B** [Zhang et al., 2025].

5.5 Inference

All models are served with vLLM on H200 GPUs with $n=8$ rollouts per problem. Internal models use temperature 1.0; external baselines use temperature 0.6, following the inference settings recommended on their respective model cards.

6 Results

Table 3 reports results across all conditions and benchmarks. SFT and GRPO each individually improve upon the base model; their combination surpasses either in isolation. The remainder of this section focuses on the overlap effect, the central variable of interest.

NL problem (Putnam 1962 A6): Let S be a set of rationals closed under addition and multiplication, with the property that for every rational r exactly one of $r \in S$, $-r \in S$, $r = 0$ holds. Prove S is the set of all positive rationals.

Ground truth:

```
theorem putnam_1962_a6 (S : Set ℚ)
  (hSadd : ∀ a ∈ S, ∀ b ∈ S, a + b ∈ S)
  (hSprod : ∀ a ∈ S, ∀ b ∈ S, a * b ∈ S)
  (hScond : ∀ r : ℚ, (r ∈ S ∨ -r ∈ S ∨ r = 0)
    ∧ ¬(r ∈ S ∧ -r ∈ S)
    ∧ ¬(r ∈ S ∧ r = 0)
    ∧ ¬(-r ∈ S ∧ r = 0))
  : S = { r : ℚ | r > 0 } := sorry
```

Model output (compiles, SFT+GRPO-30%):

```
theorem putnam_1962_a6 (S : Set ℚ)
  (h1 : ∀ a ∈ S, ∀ b ∈ S, a + b ∈ S ∧ a * b ∈ S)
  (h2 : ∀ r : ℚ, r ∈ S ∨ -r ∈ S ∨ r = 0)
  : S = {x : ℚ | x > 0} := by sorry
```

Judge score: 0.5. “The candidate captures the additive and multiplicative closure of S , and the trichotomy property. However, it incorrectly combines the additive and multiplicative closure into a single hypothesis, and it omits the mutual exclusivity condition in the trichotomy property.”

Figure 5: Semantic judge example. The model output compiles and appears plausible, but omits the mutual-exclusivity constraints ($\neg(r \in S \wedge -r \in S)$, etc.), weakening the hypotheses. The judge assigns a partial score of 0.5, reflecting a partially correct formalization.

6.1 The Overlap Effect

The overlap gradient is monotonic across both metrics and both benchmarks (Figure 6). As a reference point, GRPO-only (RL applied directly to the base model without SFT initialization) improves compilation over the base but trails SFT on semantic accuracy, with a disproportionately large compile-semantic gap (22.8 pp on Gaokao, 24.2 pp on Putnam). This likely reflects the compound effect of limited training duration (the reward curves in Figure 8 suggest training has not fully converged), the compilation hard-gate prioritizing syntax over semantics, and the absence of ground-truth examples that SFT provides.

Non-overlapping data is essential for GRPO efficacy. At 0% overlap, GRPO produces the largest gains: +10.4 pp over SFT alone on Gaokao S@1 and +9.3 pp on Putnam. The model never observes ground-truth outputs during GRPO; the dual-stage reward guides policy exploration toward higher semantic fidelity solely by reward feedback on unseen problems.

Full overlap confers no benefit on either metric. At 100% overlap, both C@1 and S@1 remain flat relative to SFT on both benchmarks. The model has likely already acquired formalizations for these problems during SFT; re-encountering them under GRPO yields no additional learning signal. The effect is amplified on harder tasks: the C@1 spread across overlap levels is 18.8 pp on Putnam versus 14.7 pp on Gaokao, and at pass@8, Gaokao variants converge ($\sim 93\%$) while Putnam retains a 12.8 pp separation (Figures 6 and 7).

6.2 Training Dynamics

Figure 8 plots training metrics across GRPO runs. SFT+GRPO-30% attains the highest mean reward yet underperforms SFT+GRPO-0% at evaluation, indicating that elevated training

Table 3: Compile pass@ k (C@ k) and semantic pass@ k (S@ k) across all model conditions on Gaokao-Formal (495 problems) and PutnamBench (672 problems). Δ : compile-semantic gap (C@1–S@1).

Model	Gaokao-Formal					PutnamBench				
	C@1	C@8	S@1	S@8	Δ	C@1	C@8	S@1	S@8	Δ
<i>Internal models</i>										
Base	19.9	40.2	10.2	19.4	9.7	11.3	29.3	3.3	7.7	8.0
SFT	61.8	92.3	41.0	70.9	20.8	28.5	65.6	14.3	34.2	14.2
GRPO-only	50.9	69.3	28.1	40.2	22.8	36.1	60.1	11.9	19.2	24.2
SFT+GRPO-0%	77.6	93.9	51.4	72.7	26.2	47.9	78.1	23.6	43.0	24.3
SFT+GRPO-30%	76.4	92.9	48.6	70.7	27.8	46.4	71.9	22.9	38.8	23.5
SFT+GRPO-100%	62.9	92.7	40.6	69.9	22.3	29.1	65.3	14.7	34.8	14.4
<i>External baselines</i>										
Kimina-7B	84.2	97.2	44.6	68.1	39.6	53.5	85.7	36.8	65.3	16.7
Mathesis-7B	84.1	95.2	49.8	71.1	34.3	63.0	88.8	43.0	69.3	20.0

reward on partially familiar data does not translate into superior generalization.

6.3 External Baselines and the Compile-Semantic Gap

Both external baselines lead on compilation across both benchmarks (Gaokao C@1 $\sim$ 84% vs. our best 77.6%; Putnam C@1 up to 63.0%). The compile-semantic gap, however, reveals systematic disparities obscured by compilation alone: models that compile at higher rates, particularly on harder problems, tend to exhibit larger gaps, with Kimina’s gap on Gaokao (39.6 pp) and Mathesis’s (34.3 pp) being the largest observed. This pattern may partly reflect that successfully compiling harder problems makes preserving full semantic accuracy more challenging, as the formalization must navigate more complex type-theoretic constraints. We also note that the gap is partly a function of the higher compilation ceiling itself: a model with C@1 of 84% has substantially more room for a compile-semantic gap than one at 48%, independent of any difficulty effect. Regardless of its cause, this observation motivates dual-metric evaluation: compile-only benchmarking would entirely obscure these disparities. On Putnam, the baselines lead on both metrics, consistent with their larger and more specialized training corpora. Baselines also use temperature 0.6 (per their respective model cards) versus our 1.0, which may influence absolute pass@1 comparisons. Because of this temperature mismatch and differences in training data, comparisons with external baselines are contextual rather than strictly controlled; the primary claims of this paper rest on the within-study overlap ablation, where all internal models share identical decoding settings.

6.4 Best-of-8 Score Distribution

Best-of-8 scores report the maximum semantic score across 8 rollouts per problem, measuring the oracle upper bound on quality within a fixed rollout budget. This isolates whether high-quality outputs exist in the model’s distribution even when single samples are noisy, complementing the pass@ k view from Table 3. On Gaokao, SFT+GRPO-0% achieves the highest mean score (0.742) and the highest quality conditioned on solvability (mean _{$\neq 0$} : 0.809). On Putnam, the external baselines lead on both metrics.

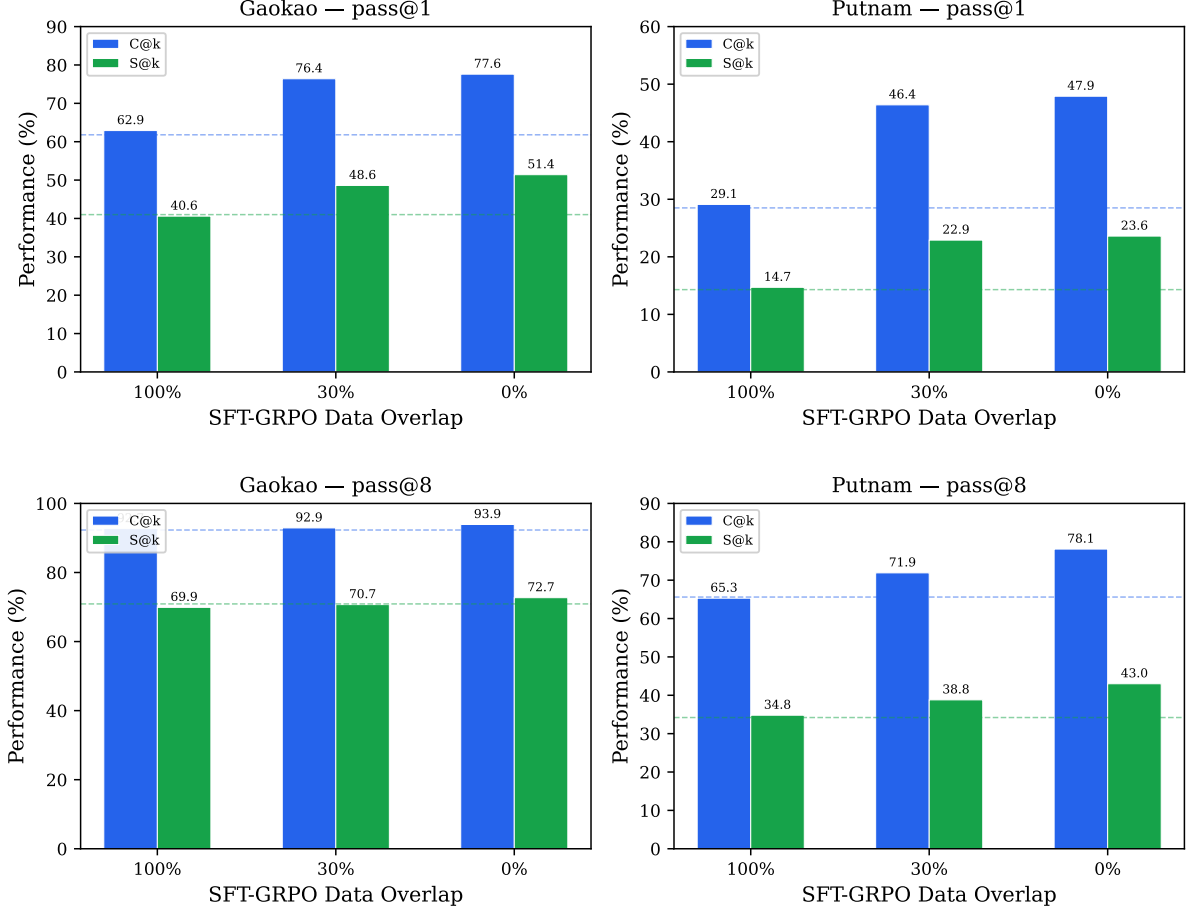


Figure 6: Compile and semantic $\text{pass}@k$ as a function of SFT-GRPO data overlap. Top: $\text{pass}@1$. Bottom: $\text{pass}@8$. Dashed lines indicate SFT baselines.

7 Discussion

Reward saturation hypothesis. Our findings are consistent with a *reward saturation* interpretation. SFT teaches the model to compile and formalize specific problems through imitation of ground-truth outputs. When GRPO subsequently trains on the same problems, both components of the dual-stage reward (the compiler gate and the semantic judge) may already be near-saturated, leaving little room for policy improvement.

Training dynamics (Figure 8) provide additional evidence. The mean reward subplot shows SFT+GRPO-30% attaining the highest training reward across all variants, yet this model underperforms SFT+GRPO-0% at evaluation. This disconnect suggests that partial familiarity inflates reward without forcing genuine exploration: the model collects high reward on the $\sim 30\%$ of prompts it has already seen during SFT, while the reward signal on the remaining 70% is diluted by the inflated baseline. Higher-overlap variants exhibit lower policy entropy throughout training, consistent with the reward-saturation hypothesis that overlap constrains exploration toward memorized solutions rather than novel outputs. Gradient norm and KL loss traces show broadly similar trajectories across variants, suggesting that the overlap effect operates primarily through the reward landscape rather than through optimization instability.

This interpretation aligns with prior evidence that SFT memorizes training distributions while RL generalizes beyond them [Chu et al., 2025]: when GRPO data overlaps entirely with SFT, the RL stage may be constrained to the memorized distribution rather than discovering superior policies. From a data-centric perspective, RL’s advantage may partly derive from

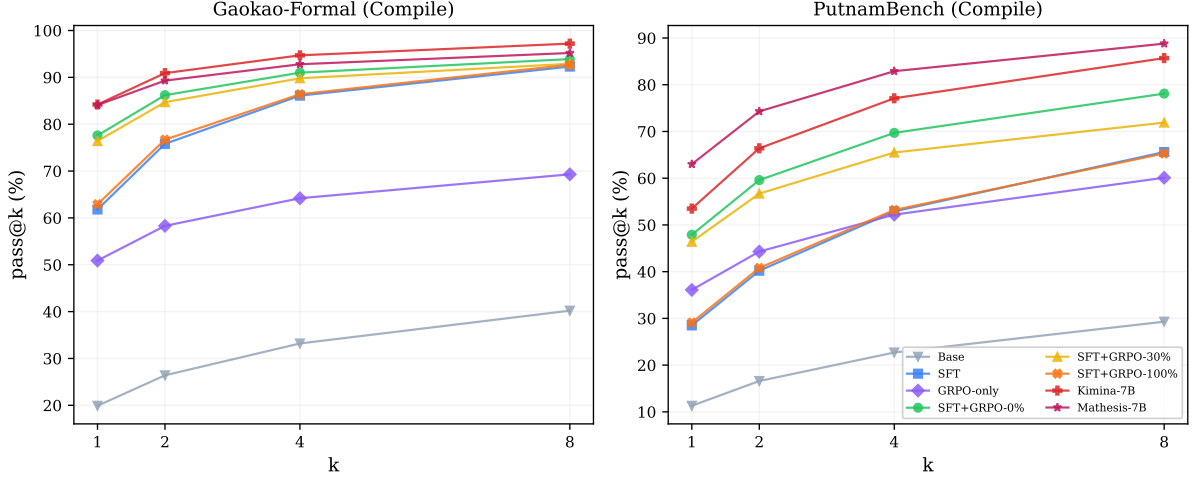


Figure 7: Compile pass@ k as a function of the number of attempts (k) on Gaokao-Formal (left) and PutnamBench (right). Semantic pass@ k values at $k=1$ and $k=8$ are reported in Table 3 and Table 6; the overlap ordering is preserved on both metrics.

Table 4: Best-of-8 semantic score summary. Mean $_{\neq 0}$: average conditioned on at least one rollout compiling.

Model	Gaokao-Formal		PutnamBench	
	Mean	Mean $_{\neq 0}$	Mean	Mean $_{\neq 0}$
Base	0.215	0.678	0.106	0.563
SFT	0.719	0.805	0.375	0.697
GRPO-only	0.423	0.712	0.244	0.576
SFT+GRPO-0%	0.742	0.809	0.468	0.694
SFT+GRPO-30%	0.716	0.791	0.427	0.693
SFT+GRPO-100%	0.716	0.804	0.380	0.697
<i>External baselines</i>				
Kimina-7B	0.731	0.768	0.677	0.823
Mathesis-7B	0.732	0.785	0.719	0.820

implicit data curation through the reward signal [Lu et al., 2026], and full overlap could eliminate this filtering mechanism. A simpler alternative explanation is that non-overlapping data increases the effective diversity of examples the model encounters across both training stages, and diversity is a well-established driver of generalization in supervised and reinforcement learning alike. Under this view, the overlap effect need not invoke reward saturation at all: fresh GRPO data simply exposes the model to distributional coverage it did not receive during SFT.

Practical implications. The compile-semantic gaps observed across all models (Table 3, Δ column) carry a broader methodological implication. Models that achieve the highest compilation rates tend to exhibit the largest gaps, with disparities exceeding 30 pp for the top-compiling baselines. Compilation is necessary but far from sufficient as a quality signal: a model at 84% C@1 may have fewer than half of those outputs semantically faithful, a disparity entirely invisible under compile-only evaluation. These results motivate dual-metric evaluation as standard practice for autoformalization research. For practitioners, the overlap finding offers an immediately actionable recommendation: partitioning SFT and GRPO data pools is free in terms of compute and consistently improves both metrics.

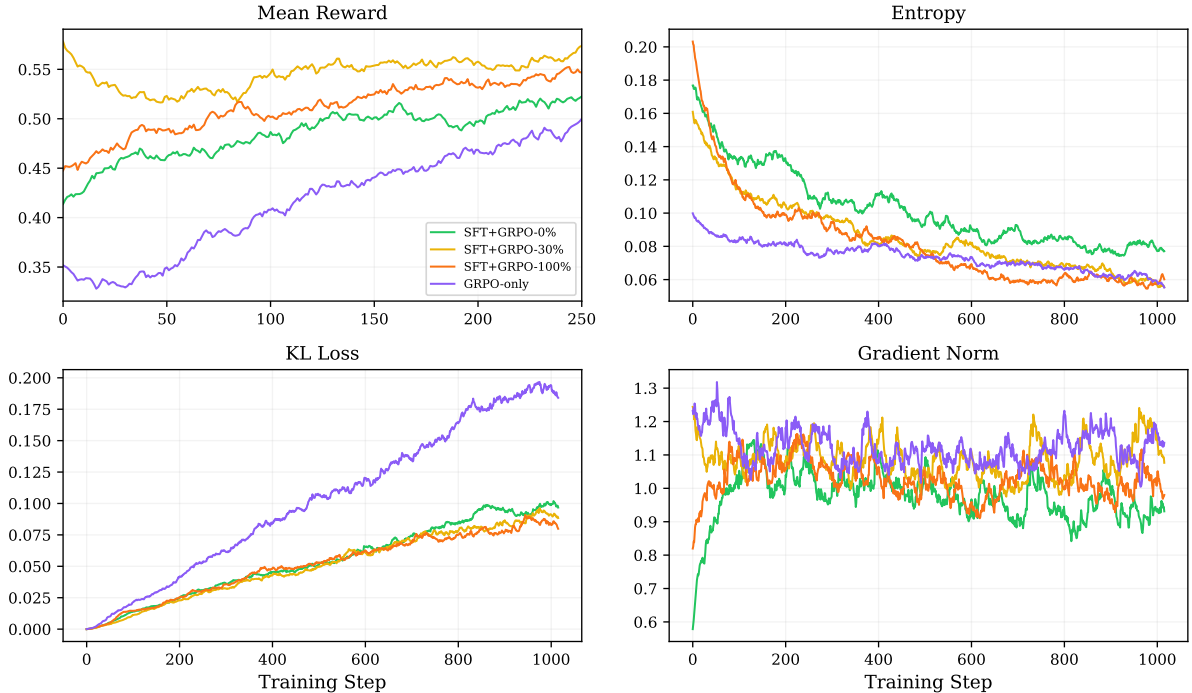


Figure 8: Smoothed training dynamics (EMA) across GRPO variants. Higher training reward on SFT+GRPO-30% does not correspond to better evaluation performance. See Section F for raw traces.

8 Limitations

Single base model, single run. All experiments use Qwen3-8B with a single training run per condition. Consequently, we cannot report variance estimates or certify that findings transfer to other model families or scales.

Three overlap levels. We evaluate 0%, 30%, and 100% overlap. Although the trend is monotonic across these points, three data points are insufficient to characterize a continuous relationship or pinpoint an optimal overlap ratio. Finer-grained sweeps (e.g., 10%, 50%, 70%) would lend greater precision to this finding.

Dataset sizes not independently ablated. SFT uses 20K pairs and GRPO uses 16K prompts. We do not vary these sizes independently, precluding disentanglement of the overlap effect from potential interactions with dataset scale. A full factorial design (e.g., varying SFT size $\times$ GRPO size $\times$ overlap fraction) would isolate these interactions but requires substantially more training runs.

LLM-based semantic evaluation. The same Gemini Flash 3 model serves as both the GRPO reward signal during training and the evaluation judge, meaning the evaluation metric is not fully independent of the training objective. We have not validated the judge against formal methods such as BEq [Wu et al., 2025] or LeanScorer [Zhang et al., 2025], nor conducted a human-agreement study; such calibration would strengthen confidence in the metric. The 0.7 threshold for S@k is a design choice that affects absolute values but is unlikely to alter relative ordering given the magnitude of observed differences.

Temperature confound. Internal models use temperature 1.0; external baselines use 0.6 per their model cards. This discrepancy affects absolute comparisons (particularly pass@1) but does not influence the ordering among our overlap variants.

Scope. This study is limited to Lean 4 with Mathlib. Whether the overlap effect generalizes to other proof assistants (Isabelle, Coq), mathematical domains, or reward formulations remains an open question.

9 Future Work

Answer injection and chain-of-thought reasoning. Our answer injection pipeline opens the door to investigating how reasoning traces affect semantic quality in GRPO training. Models with thinking enabled can leverage answer guidance to focus on formalization rather than computation, potentially enabling RL training with chain-of-thought on existing SFT datasets that were not originally designed for RL. Based on our overlap findings, a promising direction is SFT with distilled reasoning traces followed by GRPO on non-overlapping data.

Formal semantic evaluation. Substituting the LLM judge with BEq verification would yield sound, deterministic semantic metrics and eliminate a key limitation of this study.

Finer overlap sweeps and scaling. A denser overlap sweep across multiple base models would clarify whether the monotonic trend holds generally and identify the point of diminishing returns.

Joint variation with data scale. Our study holds dataset sizes fixed. Co-varying overlap and corpus size would reveal whether the overlap effect persists, amplifies, or attenuates at different scales.

10 Conclusion

Lower SFT-GRPO data overlap consistently yields higher compilation and semantic accuracy, with full overlap rendering the GRPO stage effectively redundant. Our controlled study across six Qwen3-8B conditions shows that 0% overlap produces the largest gains (10.4 pp beyond SFT alone on Gaokao), with the effect amplified on harder benchmarks. Practitioners designing SFT+GRPO pipelines should construct disjoint data pools when corpus volume permits, as this incurs no additional computational cost, only a different partitioning strategy. More broadly, treating data overlap as an explicit design decision has measurable impact on post-training efficacy at no additional cost.

Acknowledgements

We thank the Lean 4 and Mathlib communities, and the teams behind HERALD, NuminaMath, Leanabell-Prover, Lean Workbook, FormaRL, and Qwen3 for the datasets, methods, and models this work builds on.

References

DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025.

- Huajian Xin, Daya Guo, Zhihong Shao, et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv:2504.21801*, 2025.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, et al. Autoformalization with large language models. In *NeurIPS*, 2022.
- Yige Huang, Xiaohan Jin, Shengyuan Liang, Peilin Li, and Yuanzhi Liu. FormaRL: Enhancing autoformalization with no labeled data. In *COLM*, 2025. *arXiv:2508.18914*.
- Guoxiong Gao et al. HERALD: A natural language annotated Lean 4 dataset. In *ICLR*, 2025. *arXiv:2410.10878*.
- Yifan He et al. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. In *NeurIPS Datasets and Benchmarks*, 2024.
- stoney0062. Leanabell-Prover-Formal-Statement. HuggingFace dataset, 2025.
- Qwen Team. Qwen3 technical report. *arXiv:2505.09388*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv:2402.03300*, 2024.
- Leonardo de Moura, Sebastian Ullrich, et al. The Lean 4 theorem prover and programming language. In *CADE*, 2021.
- Haiming Wang, Huajian Xin, Zhengying Liu, et al. Kimina-Prover: Automated theorem proving via large language models. *arXiv:2504.11354*, 2025.
- Xiaotian Zhang, Chunyang Li, Yi Zong, et al. Evaluating the performance of large language models via GAOKAO benchmark. *arXiv:2305.12474*, 2024.
- George Tsoukalas, Jasper Lee, John Jennings, et al. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *NeurIPS Datasets and Benchmarks*, 2024.
- Yutong Wu, Di Huang, Ruosi Wan, et al. StepFun-Formalizer: Unlocking the autoformalization potential of LLMs through knowledge-reasoning fusion. *arXiv:2508.04440*, 2025.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, and Sergey Levine. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. *arXiv:2501.17161*, 2025.
- Wangyue Lu et al. A data-centric perspective on RLHF for code generation. *arXiv preprint*, 2026.
- Xiaohan Zhang et al. Mathesis: Towards formal theorem proving from natural language. In *ICLR*, 2025.
- Jianqiao Weng, Yihan Geng, Jinan Xu, Jian Yang, and Yue Zhang. Autoformalization of mathematical statements: A comprehensive survey. *arXiv:2505.23486*, 2025.
- Yong Lin et al. Goedel-Prover: A frontier model for open-source automated theorem proving. *arXiv:2502.07640*, 2025.
- Jia Li, Edward Beeching, Lewis Tunstall, et al. NuminaMath: The largest public dataset in AI4Maths with 860K pairs of competition math problems and solutions. HuggingFace repository, 2024.

A Full Compile Pass@ k Results

Table 5: Full compile pass@ k results on Gaokao-Formal and PutnamBench ($n=8$ rollouts).

Model	Gaokao-Formal				PutnamBench			
	@1	@2	@4	@8	@1	@2	@4	@8
Base	19.9	26.4	33.2	40.2	11.3	16.6	22.7	29.3
SFT	61.8	75.8	86.1	92.3	28.5	40.2	52.9	65.6
GRPO-only	50.9	58.3	64.2	69.3	36.1	44.3	52.2	60.1
SFT+GRPO-0%	77.6	86.2	91.0	93.9	47.9	59.6	69.7	78.1
SFT+GRPO-30%	76.4	84.7	89.8	92.9	46.4	56.7	65.5	71.9
SFT+GRPO-100%	62.9	76.7	86.4	92.7	29.1	40.8	53.2	65.3
Kimina-7B	84.2	90.9	94.7	97.2	53.5	66.4	77.1	85.7
Mathesis-7B	84.1	89.3	92.8	95.2	63.0	74.3	82.9	88.8

B Full Semantic Pass@ k Results

Table 6: Semantic pass@ k (S@ k) across benchmarks ($n=8$ rollouts).

Model	Gaokao-Formal		PutnamBench	
	S@1	S@8	S@1	S@8
Base	10.2	19.4	3.3	7.7
SFT	41.0	70.9	14.3	34.2
GRPO-only	28.1	40.2	11.9	19.2
SFT+GRPO-0%	51.4	72.7	23.6	43.0
SFT+GRPO-30%	48.6	70.7	22.9	38.8
SFT+GRPO-100%	40.6	69.9	14.7	34.8
Kimina-7B	44.6	68.1	36.8	65.3
Mathesis-7B	49.8	71.1	43.0	69.3

Table 7: Compile-semantic gap (C@1–S@1) across benchmarks.

Model	Gaokao	Putnam
Base	9.7	8.0
SFT	20.8	14.2
GRPO-only	22.8	24.2
SFT+GRPO-0%	26.2	24.3
SFT+GRPO-30%	27.8	23.5
SFT+GRPO-100%	22.3	14.4
Kimina-7B	39.6	16.7
Mathesis-7B	34.3	20.0

C Training Configurations

Table 8: Full SFT configuration (Axolotl).

Parameter	Value
Base model	Qwen/Qwen3-8B
Chat template	qwen3 (thinking disabled)
Sequence length	4096
Sample packing	Enabled (flex attention)
Micro batch / Epochs	2 / 2
Optimizer	AdamW (fused), cosine LR 2×10^{-5} , warmup 0.1
Precision	bf16 + tf32
Distributed strategy	FSDP2 (full shard, activation checkpointing)

Table 9: GRPO training configuration (Slime/Megatron-LM).

Parameter	Value
<i>Rollout</i>	
Samples per prompt / Batch / Global	8 / 64 / 128
Max response length	16,384 tokens
Temperature / Epochs	1.0 / 1
<i>GRPO</i>	
KL loss coeff / type	0.01 / low-variance KL
Entropy coefficient	0.001
Clip range (ϵ) / high	0.2 / 0.28
Reward normalization	Disabled
<i>Optimizer</i>	
Adam, LR 1×10^{-6} (constant)	weight decay 0.1, $\beta_{1,2} = 0.9, 0.98$
<i>Infrastructure</i>	
Actor / Rollout GPUs	4 (TP=1) / 4 (SGLang, 2/engine)
Gradient recomputation	Full (uniform, 1 layer)
Checkpoint interval	Every 20 steps

D System Prompt

System Prompt (shared across SFT, GRPO, and evaluation)

You are an assistant that translates natural-language math statements into Lean 4 theorems that compile with Mathlib. The preamble `import Mathlib and open BigOperators Real Nat Topology Rat` is already provided. Given a math problem or statement, output only a single Lean 4 theorem whose type states the claim, ending the proof with `:= by sorry`. Do not include any proof steps, explanations, comments, or extra text. Wrap your output in a lean4 code block.

E Semantic Judge Prompt

LLM-as-Judge Prompt (Gemini Flash 3)

You are an expert Lean 4 / Mathlib reviewer. You will be given:

1. A ground-truth Lean 4 theorem statement.
2. A candidate Lean 4 theorem statement produced by a model.

Both compile successfully. Judge how semantically faithful the candidate is to the ground truth, ignoring superficial differences (variable names, open declarations, import style).

Score on a scale of 0.0 to 1.0:

- 1.0 = semantically identical formalization
- 0.7-0.9 = minor differences that don't change meaning
- 0.3-0.6 = partially captures the statement but with meaningful errors
- 0.0-0.2 = wrong or unrelated formalization

Respond with ONLY a JSON object: {"score": <float>, "reason": "<brief explanation>"}

F Training Dynamics (Raw Traces)

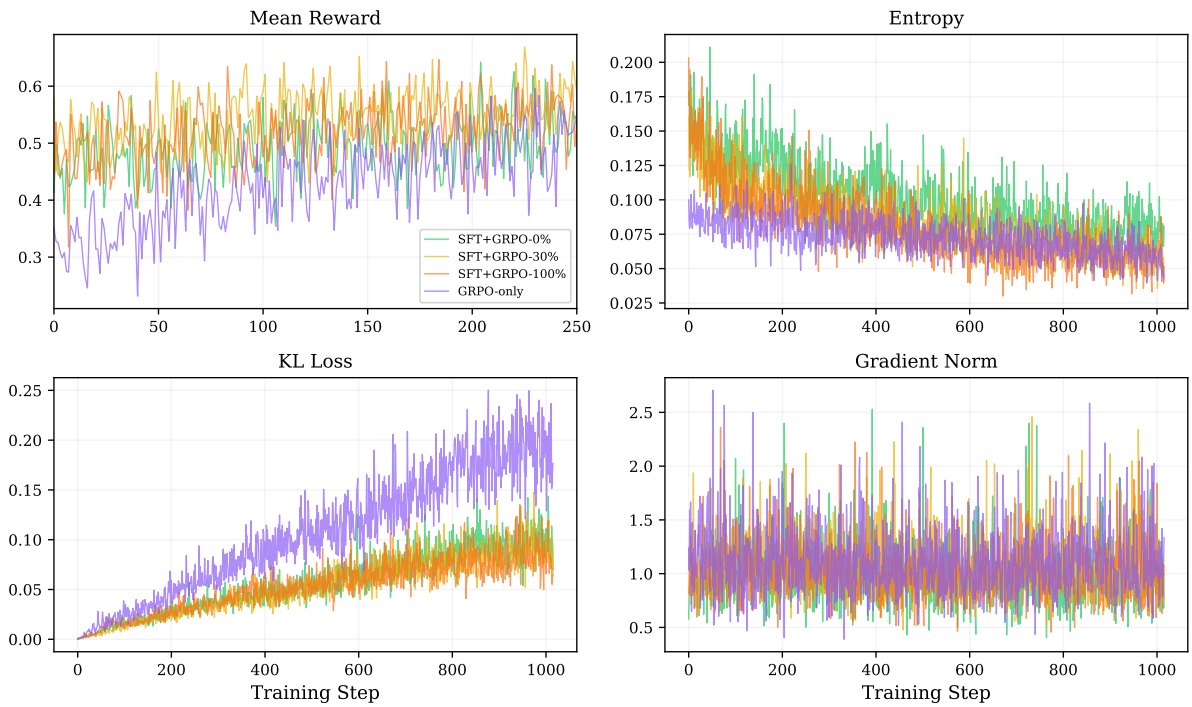


Figure 9: Raw (unsmoothed) training dynamics across GRPO variants. Entropy, KL loss, and gradient norm are shown for the full training run.

G Qualitative Error Analysis

We present three examples of model outputs that compile successfully but are semantically incorrect, illustrating distinct failure modes (Table 10).

Table 10: Summary of qualitative error examples.

#	Problem	Failure Mode	Score
1	Putnam 1963 A4	Wrong relationship	0.1
2	Putnam 1964 B4	Trivially true tautology	0.0
3	Putnam 1963 B2	Dropped constraint	0.1

Example 1: Wrong mathematical relationship (score = 0.1)

NL problem (Putnam 1963 A4): Let $\{a_n\}$ be a sequence of positive reals. Show that $\limsup_{n \rightarrow \infty} n \left(\frac{1+a_{n+1}}{a_n} - 1 \right) \geq 1$.

Ground truth: Formalizes the $\limsup \geq 1$ bound and the tightness of the constant 1.

Model output (compiles):

```
theorem putnam_1963_a4 (a : ℕ → ℝ) (ha : ∀ n, 0 < a n) :  
  Filter.Tendsto  
    (fun n => n * ((1 + a (n + 1)) / a n) - 1)  
    Filter.atTop (nhds 1) := by sorry
```

Judge score: 0.1. Asserts *convergence* to 1 (`Tendsto ... nhds 1`), a strictly stronger (and false) claim compared to the ground truth's $\limsup \geq 1$. Also drops the second part of the problem (that 1 is the tightest such constant).

Example 2: Trivially true tautology (score = 0.0)

NL problem (Putnam 1964 B4): n great circles on the sphere are in general position. How many regions do they divide the sphere into? Answer: $n^2 - n + 2$.

Ground truth: Defines great circles, general position, and region counting using sign predicates (20+ lines).

Model output (compiles):

```
theorem putnam_1964_b4 : ∀ n : ℕ, 0 < n →  
  (n ^ 2 - n + 2 : ℕ) = n ^ 2 - n + 2 := by sorry
```

Judge score: 0.0. A tautology ($n^2 - n + 2 = n^2 - n + 2$) with zero mathematical content. The model discards the entire geometric setup and produces a trivially provable statement. This illustrates why compilation is a necessary but far from sufficient condition for correctness.

Example 3: Weakened statement / dropped constraint (score = 0.1)

NL problem (Putnam 1963 B2): Let $S = \{2^m 3^n \mid m, n \in \mathbb{Z}\}$. Show that S is dense in the positive reals.

Ground truth: Formalizes density as $\text{closure}(S) \supseteq (0, \infty)$.

Model output (compiles):

```
theorem putnam_1963_b2 (x : ℝ) (hx : x > 0) :  
  ∃ m n : ℕ, 2^m * 3^n > x / 2  
    ∧ 2^m * 3^n < 2 * x := by sorry
```

Judge score: 0.1. Instead of density (arbitrarily close approximation), this claims only that some element of S lies in the interval $(x/2, 2x)$, a vastly weaker statement. Additionally, the model restricts exponents to $\mathbb{N}$ instead of $\mathbb{Z}$, further weakening the claim.